Chapter 3 Processing and Understanding Text

Word Tokenization

Tokenization can be defined as the process of breaking down or splitting textual data into smaller and more meaningful components called *tokens*.

First, a Toy Data Set

```
import pandas as pd
import numpy as np
```

```
sentence_list = ["First, we consider the initial spreadsheet model 123@.",
                 "Then we consider another model @247, not the initial one.",
                 "First, we consider the initial spreadsheet model "
                 "and then we consider another model. So 2."]
```

```
sentences_df = pd.DataFrame(sentence_list, columns = ["sentence"])
sentences_df
```

```
pd.set_option('max_colwidth', 400)
```

```
sentences_df
```

```
sample_text = sentences_df.sentence.str.cat(sep=' ')
sample_text
```

NLTK

Natural Language Toolkit

https://www.nltk.org/

"NLTK is a leading platform for building Python programs to work with human language data. It provides easy-to-use interfaces to over 50 corpora and lexical resources such as WordNet, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning, wrappers for industrial-strength NLP libraries, and an active discussion forum."

```
import nltk
```

Regex Word Tokenizer

```
TOKEN_PATTERN = r'\w+'
regex_wt = nltk.RegexpTokenizer(pattern=TOKEN_PATTERN, gaps=False)
```

```
words = regex_wt.tokenize(sample_text)
np.array(words)
```

```
GAP_PATTERN = r'\s+'
regex_wt = nltk.RegexpTokenizer(pattern=GAP_PATTERN, gaps=True)
```

```
words = regex_wt.tokenize(sample_text)
np.array(words)
```

```
word_indices = list(regex_wt.span_tokenize(sample_text))
word_indices
```

```
np.array([sample_text[start:end] for start, end in word_indices])
```

Derived Regex Tokenizers

The WordPunktTokenizer uses the pattern r'\w+|[^\w\s]+' to tokenize sentences into independent alphabetic and non-alphabetic tokens.

```
wordpunkt_wt = nltk.WordPunctTokenizer()
words = wordpunkt_wt.tokenize(sample_text)
```

```
np.array(words)
```

The WhitespaceTokenizer tokenizes sentences into words based on whitespace, like tabs, newlines, and spaces.

```
whitespace_wt = nltk.WhitespaceTokenizer()
words = whitespace_wt.tokenize(sample_text)
```

```
np.array(words)
```

Cleaning Text

```
from string import punctuation
print(punctuation)
```

```
nltk.download('stopwords')
```

```
from nltk.corpus import stopwords
```

```
stop_words = set(stopwords.words('english'))
print(stop_words)
```

Cleaning Text Function

```
# clean text
def clean_text(text):
    # split on white spaces
    tokens = text.split()
    # remove punctuation
    # If there is a third argument, it must be a string,
    # whose characters will be mapped to None in the result.
    table = str.maketrans('','',punctuation)
    tokens = [word.translate(table) for word in tokens]
    # remove non-alphabetic
    tokens = [word for word in tokens if word.isalpha()]
    # change all words to lower case
    # (depending on which tokenizer you use,
    # it would do that by default, too)
    tokens = [word.lower() for word in tokens]
    # filter out stopwords
    tokens = [word for word in tokens if not word in stop_words]
    # filter out short words
    tokens = [word for word in tokens if len(word) > 1]
    return ' '.join(tokens)
```

Apply the Function

```
print(clean_text(sample_text))
```

```
# clean sentences
sentences_df.sentence = sentences_df.sentence.apply(clean_text)
sentences_df
```

Stemming

Idea: reduce words to their **root stem** via rules, may **not** be *valid* words.

Pros: fast, shrinks vocabulary

Cons: can be aggressive (*better* → *bet* with Lancaster)

Common stemmers: Porter (gentler), Lancaster (aggressive), Snowball (Porter2)

Stemming in Python

```
#  NLTK stemmers (Porter, Lancaster)
from nltk.stem import PorterStemmer, LancasterStemmer
```

```
ps, ls = PorterStemmer(), LancasterStemmer()
```

```
tokens = ["running", "jumps", "better", "studies", "studying", "happiest"]
```

```
print("Porter:",   [ps.stem(t) for t in tokens])
print("Lancaster:", [ls.stem(t) for t in tokens])
```

Lemmatization

Idea: map words to their **dictionary base form (lemma)** using a lexicon + Part Of Speech (POS).

Pros: produces real words (run, good, study)

Cons: slower, needs POS/lexicon

Tools: NLTK WordNetLemmatizer

Lemmatization in Python

```
# NLTK lemmatizer (WordNet)
nltk.download('wordnet')
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet
```

```
wnl = WordNetLemmatizer()
```

```
# n = noun, v = verb, a = adjective, r = adverb
examples = [("running", "v"), ("better", "a"), ("studies", "n"), ("studying",
"v")]
```

```
print([ (w, pos, wnl.lemmatize(w, pos=pos)) for (w, pos) in examples ])
```

Stemming vs. Lemmatization

Stemming: fast heuristic; good for quick BoW/TF-IDF experiments

Lemmatization: cleaner vocab; better for downstream semantics

Both reduce sparsity: *run, runs, running* → **run**; but only lemmatization maps *ran* → **run**

Additional Readings

Some ideas based on https://machinelearningmastery.com/prepare-movie-review-data-sentiment-analysis/

Text Analytics with Python: A Practitioner's Guide to Natural Language Processing Paperback, 2019, by Dipanjan Sarkar